

Understanding Request and Response

The Foundation of Web Communication

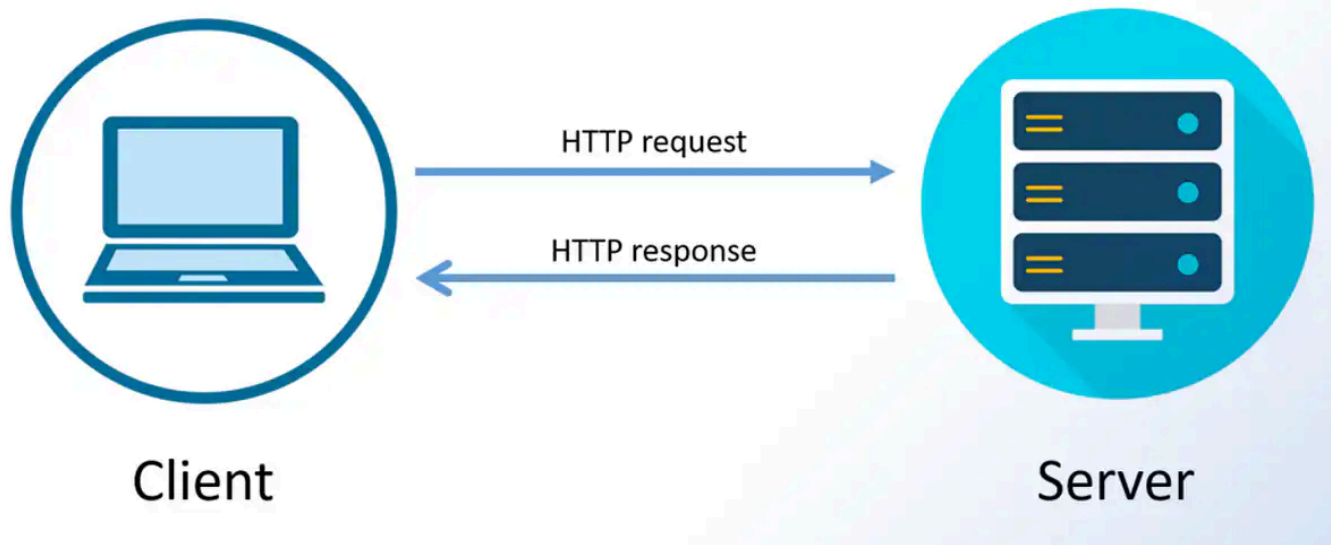
Learning Goals

- Define HTTP requests and responses
- Explain why manually parsing HTTP is complex
- Identify the risks and costs of manual HTTP parsing
- Match common request data to PHP access mechanisms

IMPORTANT

What is HTTP **Request** and **Response** ?

- **HTTP (HyperText Transfer Protocol)** is the foundation of web communication
- **Request:** Client (browser/curl) asks for something from the server
- **Response:** Server sends back the requested data or status



| Clients

A **client** is anything that creates and sends an HTTP request to a server.

- A web **browser** is a client when you visit a website.
- The `curl` command is also a client when you send a request from a terminal.
- Other examples include mobile apps, JavaScript code, and API testing tools.

The following HTTP request can be created by **any of these clients**.

Raw HTTP **Request** Format

When a client sends a request, the raw HTTP message can look like this:

```
POST /api HTTP/1.1
Host: localhost:8000
Content-Type: application/json

{
  "student_id": 1
}
```

Notice the space between headers and body.

Components

- **Request Line:** Method + Path + HTTP Version
- **Headers:** Metadata about the request
- **Body:** Data (for POST/PUT requests)

Raw HTTP **Response** Format

The server responds with something like this:

```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 285
Date: Sat, 02 Aug 2025 10:30:00 GMT
Server: PHP Development Server

{
  "success": true,
  "message": "Welcome to Simple PHP API",
  "data": {
    "name": "Simple PHP API for Education",
    "version": "1.0"
  }
}
```


Components:

- **Status Line:** HTTP Version + Status Code + Status Message
- **Headers:** Metadata about the response
- **Body:** The actual content

IMPORTANT

Why Manual Parsing is Complex

- Servers must parse client requests to generate responses.
- Manual parsing is error-prone because HTTP has several details to handle.

1. Format Complexity

```
GET /search?q=hello+world&lang=en HTTP/1.1\r\n
Host: example.com\r\n
User-Agent: Mozilla/5.0...\r\n
\r\n
```

- Must handle `\r\n` line endings
- Parse query parameters manually
- Decode percent-encoding (`%20` = space); in form-style query strings, `+` also means space

2. Header Parsing Challenges

Python code that manually parses an HTTP request:

```
def parse_headers(raw_request): # 1
    lines = raw_request.split('\r\n') # 2
    request_line = lines[0].split(' ') # 3
    method = request_line[0] # 4
    path = request_line[1] # 5

    headers = {} # 6
    for line in lines[1:]: # 7
        if line == '': # 8
            break # 9
        key, value = line.split(':', 1) # 10
        headers[key.lower()] = value # 11

    return method, path, headers # 12
```

Problems:

- **Lines 3-5:** If the request line is empty or has no space (a malformed line with fewer than two tokens), `request_line[1]` raises an `IndexError` — the code assumes at least two space-separated tokens are always present.
- **Line 10:** The code requires `:`; `Host:example.com` or `InvalidHeader` cannot be split into a key and value.
- **Line 11:** Any header name and value are stored without validation, allowing unexpected or

3. URL and Query Parameter Parsing

```
/search?name=John%20Doe&age=25&city=New%20York
```

Manual parsing required:

- Split path from query string
- Decode URL encoding (`%20` → space)
- Parse key-value pairs
- Handle arrays: `?colors=red&colors=blue`
- Handle special characters

- **Path:**

`/search`

-> The endpoint/resource requested.

- **Query String:**

`name=John%20Doe&age=25&city=New%20York`

-> Key-value pairs after `?`.

- Each item is separated by `&`

- `name=John%20Doe` → `name = "John Doe"`

- `age=25` → `age = 25`

- `city=New%20York` → `city = "New York"`

4. Content Type Handling

- **JSON:** `{"name": "John"}`
- **Form data:** `name=John&age=25`
- **Multipart:** File uploads with boundaries
- **XML:** `<user><name>John</name></user>`

IMPORTANT

Result of Manual HTTP Parsing

- Hundreds of lines of complex code
- Security vulnerabilities
- Maintenance nightmare
- Reinventing the wheel

We Need a Tool to Handle This Complexity

It should:

- Parse `requests` and `responses`
- Help validate input
- Provide routing and simple variable access

IMPORTANT

PHP: The Tool We Will Use

PHP gives us request data without manual parsing:

- Query strings: `$_GET`
- Form data: `$_POST`
- Request method and headers: `$_SERVER`
- JSON body: read `php://input`, then decode it

PHP converts complex raw HTTP

```
GET /api?name=John&age=25 HTTP/1.1  
Host: localhost:8000
```

Into simple variable access through parsing:

```
$method = $_SERVER['REQUEST_METHOD']; // "GET"  
$name = $_GET['name'];                // "John"  
$age = $_GET['age'];                  // "25"
```